

Report: Behavioral Dynamics Prediction using Attention-Based Neural Point Process

Project Overview

In this project, I set out to predict forum engagement behavior using a deep learning model inspired by the paper "Modeling Behavioral Dynamics in Digital Content Consumption" (Marketing Science, 2020).

I modeled three behavioral components:

- $\pi(t)$: Whether a reply will occur (binary classification)
- $\lambda(t)$: When it will happen (regression in hours)
- **count(t)**: How many replies will occur in the next 24 hours (multi-class classification)

How I Referred to the Paper

The original paper proposed an attention-based neural point process to model user behavior on platforms like video games and news websites. I adapted their architecture to fit my own asynchronous, post-based forum dataset:

- **GRU backbone**: To retain the sequential context of forum replies.
- **Decayed attention modules**: To mimic the temporal decay of influence from earlier posts.
- **Multi-task prediction heads**: Just like the paper predicted event occurrence, timing, and intensity, I predicted reply probability, reply time, and reply count.
- **Textual and temporal inputs**: I combined SBERT embeddings with engineered temporal and structural features to replicate the input richness described in the paper.

This mapping allowed me to bring their point-process modeling ideas into a forum-style environment, opening up the ability to reason about thread activity in a more structured way.

Dataset and Preprocessing

I worked with two CSV files:

- **sentiment_replies.csv** for post-level metadata
- **topic.csv** for thread-level metadata

Key Preprocessing Steps:

- Cleaned messy fields like `dateTime`, `postcount`, and `threadDetails`
- Merged the datasets using `connecting_id`
- Computed derived fields:
 - **delta_t**: Hours since the last reply
 - **label**: Whether a reply occurs within 24 hours
 - **lambda_target**: Log-transformed time to next reply
 - **count_target**: Replies within 24h (bucketized: 0/1/2/≥3)
- Added engineered features: `hour`, `weekday`, `position index`, `is_thread_starter`
- Used **SBERT (MiniLM)** to get 384-dim semantic embeddings of reply text

Final input dimension per post: 395 features

Model Architecture

I designed a GRU-based attention model with three task-specific decayed attention layers.

Core Modules:

- GRU: **GRU(input_dim=395, hidden_dim=128)**
- Three decayed attention modules for $\pi(t)$, $\lambda(t)$, and count(t)
- **Heads:**
 - **reply_head:** Sigmoid output
 - **time_head:** Softplus output
 - **count_head:** Softmax output over 4 classes

Loss Function:

- Binary cross-entropy for reply prediction
- MSE (log space) for time prediction
- Cross-entropy loss for count prediction

I weighed them as follows:

Total Loss = $2.0 \times \text{reply_loss} + 1.0 \times \text{lambda_loss} + 0.5 \times \text{count_loss}$

Why This Weighting Strategy?

The goal of this weighted loss strategy was to balance learning across the three tasks while recognizing their varying difficulty and importance:

Reply Prediction ($\pi(t)$) was my primary task. It had the most direct evaluation metrics (precision, recall, F1) and cleaner, less noisy labels. Giving it a higher weight (2.0) helped the model prioritize learning to predict whether a reply would happen — especially important since forum dynamics depend heavily on activity prediction.

Time Prediction ($\lambda(t)$) was treated with a standard weight (1.0). This task is inherently harder due to its regression nature and the wide variance of true values (from a few minutes to thousands of hours). By applying a log transform and giving it moderate weight, I encouraged the model to learn temporal sensitivity without letting large errors dominate the gradient.

Count Prediction (count(t)) was most affected by class imbalance — with class 0 (no replies in 24h) being dominant. To avoid the model being overwhelmed by noisy or imbalanced supervision, I assigned it a lower weight (0.5). This still allowed learning of reply volume patterns while keeping training stable.

Overall, this combination helped the model stay focused on accurate event prediction while also learning temporal and volume trends without destabilizing multitask learning.

Implementation Timeline & Challenges

Initial Setup

- Loaded both datasets
- Handled inconsistent timestamps
- Extracted numeric reply_index from threadDetails

Issue: lambda_target had extreme values

- **What I saw:** Gaps over 1000+ hours between replies
- **How I fixed it:** Applied log1p transform to stabilize magnitude

Issue: $\pi(t)$ predictions stuck at 0.5

- **What I saw:** Model lacked confidence
- **Fixes I tried:**
 - Doubled the loss weight on $\pi(t)$
 - Verified class imbalance was being learned
 - Confirmed SBERT embeddings were feeding in
- **Result:** Significant gain in F1 (up to 0.78)

Issue: $\lambda(t)$ predicted near-constant times

- **What I saw:** Flat predictions around ~0.5 hours
- **Fixes I tried:**
 - Emphasized lambda loss
 - Added decayed attention to model time decay
- **Result:**
 - Mild improvement, but spikes still underfit

Issue: count(t) always predicted class 0

- **What I saw:** Model overly biased to zero-reply class
- **Fixes I tried:**
 - Reduced target space by bucketing
 - Increased count loss weight
 - Added its own attention head
- **Result:**
 - Some improvement, but still dominated by majority class

Upgrades Over Time

- Added SBERT instead of CountVectorizer
 - Normalized numerical fields with MinMaxScaler
 - Injected hour, weekday, and positional index into features
 - Used ReduceLROnPlateau and early stopping
 - Saved checkpoints to "best_multitask_model.pt"
-

Evaluation

Visualizations

- Compared predicted vs. true values for $\pi(t)$, $\lambda(t)$, count(t)
- Noted flat outputs vs. spikes in true values

Metrics (on validation set)

- $\pi(t)$: Precision = 0.66, Recall = 0.97, F1 = 0.78
- $\lambda(t)$: MAE $\approx$ 109 hours
- **count(t)**: High bias toward 0, visible in confusion matrix

What I Want to Improve Next

Idea	Why
Use Bi-GRU	Capture future context
Try Transformers	Let model learn long-range dependencies
Add Focal Loss	Fix class imbalance in $\pi(t)$
Weight count classes	Penalize mistakes on rare classes
Use oversampling	Balance out rare post conditions

Final Thoughts

This project let me implement a research-grade neural point process model customized for forum threads. Along the way, I:

- Designed a custom multi-headed attention model
- Used SBERT and engineered features together
- Overcame cleaning, bias, and convergence issues
- Evaluated all three prediction heads in tandem

There's more room for tuning and experimentation, especially with $\lambda(t)$ and count(t), but I now have a solid and modular baseline to extend from. I'm proud of how closely I aligned this work with a published research paper and transformed it into a working, forum-focused solution.